
CLUSTER

Backend Upgrade Strategy

From MVP (74%) to Production-Grade Stablecoin Rails

Confidential — Internal Use Only | May 2026

Executive Summary

After a deep audit of the Django Blockchain Engine (18 models, 25+ endpoints, 4 chains), the Next.js API proxy layer (52 route files), and the PRD scorecard, the backend is architecturally sound but operationally incomplete. The upgrade focuses on five pillars: Transaction Integrity, Security Hardening, Observability, Testing, and Scalability.

Total timeline: 14 weeks (3.5 months) with 1-2 backend engineers.

Current State Assessment

Layer	Grade	Notes
Design System	A-	Cohesive auth + dashboard, CSS tokens, dark mode, mobile
Auth Flow	B+	Firebase + 2FA + OTP, multi-step signup, social login scaffolded
Frontend Architecture	B+	Next.js 16, Zustand, React Query, graceful degradation
Exchange Rates	B	CoinGecko + forex, dual premium (2%/2.5%), 5-min cache
Multi-chain	B-	BSC, Solana, TRON, ETH scaffolded
KYC/Compliance	C+	BVN exists, NDPR gaps, sanctions screening unclear
Fiat Rails	C	Bank verify exists, Paystack/Flutterwave not plumbed
Transaction Engine	C	No retry, no idempotency, fake tx hashes
Security	C+	No rate limiting, webhook HMAC not enforced, no RBAC
Testing	D	Sparse unit tests, no e2e, no money-path coverage
Monitoring	D	No Sentry, no structured logging, no alerting

Phase 0 — Critical Blockers (Week 1)

These must be fixed before any user touches real money.

0.1 Replace generate_random_tx_hash()

The withdrawal endpoint generates fake tx hashes. Wire through real chain managers, return real tx_hash, set status=pending.

0.2 Enforce Webhook Signature Verification

WebhookVerifier exists but is NOT called. Any POST to /api/bsc/webhook/ can credit balances. Add HMAC verification at top of every webhook handler.

0.3 Enable Rate Limiting

Endpoint	Limit
Login/Register	5/min per IP
KYC	3/day per user
Withdrawals	10/min per user
Webhooks	100/min per IP

0.4 Configure Email Verification

Set up Resend, re-enable the email gate in login route (currently bypassed with TODO comment).

Phase 1 — Transaction Integrity (Weeks 2-3)

Ensure no user can ever lose funds due to system behavior.

1.1 Idempotency Keys

Client sends Idempotency-Key header (UUID). Middleware caches response for 24h. Prevents duplicate orders/withdrawals on retry. New model: IdempotencyRecord.

1.2 Database Transaction Wrapping

Wrap all money-movement in `@transaction.atomic` with `select_for_update()` on balance rows. Prevents partial failures and race conditions.

1.3 Transaction Confirmation Tracking

New Celery task (every 2 min): check on-chain confirmations for pending tx. Mark confirmed when threshold met. Mark failed + alert after 30 min.

1.4 Balance Reconciliation

Daily Celery task (3 AM): compare DB balance vs on-chain balance. Log discrepancies to ReconciliationLog. Alert admin. Never auto-correct.

Phase 2 — Security Hardening (Weeks 3-4)

2.1 Admin RBAC

New AdminPermission model with roles (super_admin, admin, support, viewer). Apply permission checks to all admin views. Audit every admin action.

2.2 Vault Key Management

Immediate: move vault keys to env vars. Short-term: AWS KMS key wrapping. Long-term: HSM for signing.

2.3 HMAC Request Signing

Sign Next.js→Django requests with HMAC-SHA256(timestamp + method + path + body_hash). Reject stale requests (>5 min).

2.4 Data Masking

SafeLogger class that redacts private keys, BVN, NIN from all logs.

Phase 3 — Observability (Weeks 4-5)

3.1 Structured Logging

python-json-logger (Django) + pino (Next.js). Request ID (UUID) propagated via X-Request-ID.

3.2 Sentry Integration

Both Django and Next.js. Custom breadcrumbs for blockchain RPC, KYC, exchange rate calls.

3.3 Metrics + Alerting

Prometheus/Datadog for vault balance, withdrawal duration, RPC latency, order failure rate. PagerDuty for critical alerts.

3.4 SystemAuditLog

Immutable audit record for every balance change, status transition, and admin action. Non-negotiable for financial product.

Phase 4 — Testing (Weeks 5-7)

80%+ unit coverage on money paths. Integration tests on testnet. E2E via Playwright. Load tests via k6 (100 concurrent users, verify no balance inconsistencies).

Phase 5 — Scalability (Weeks 7-9)

PgBouncer + read replicas. Redis 4-instance architecture. Celery 3-worker isolation (blockchain, notification, maintenance). RPC failover with circuit breaker (3 nodes per chain).

Phase 6 — Feature Completion (Weeks 9-11)

Paystack/Flutterwave fiat rails. Notification service (Resend + Termii + FCM). Referral system. NDPR data export.

Priority Matrix

Phase	Effort	Risk Reduction	Revenue Impact
P0: Critical Blockers	1 week	Eliminates fraud vectors	Enables launch
P1: Transaction Integrity	2 weeks	Prevents fund loss	Trust & retention
P2: Security Hardening	2 weeks	Prevents breaches	Regulatory compliance
P3: Observability	1.5 weeks	Incident response	Reduces downtime
P4: Testing	2.5 weeks	Prevents regressions	Dev velocity
P5: Scalability	2 weeks	Handles growth	10x users
P6: Features	3 weeks	Full product	Acquisition

Phase 0 is non-negotiable. Phases 1-3 before public launch. Phases 4-6 can run with controlled beta.